Exercise 2: E-commerce Platform Search Function

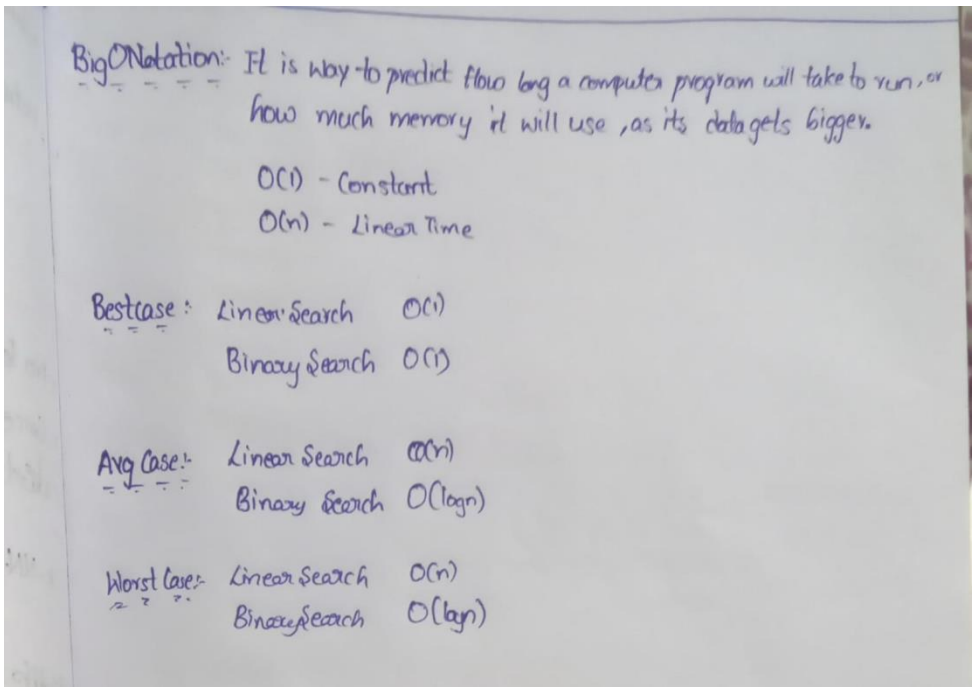
Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. **Understand Asymptotic Notation:**
    - Explain Big O notation and how it helps in analyzing algorithms.
    - Describe the best, average, and worst-case scenarios for search operations.
2. **Setup:**
    - Create a class **Product** with attributes for searching, such as **productId, productName**, and **category**.
3. **Implementation:**
    - Implement linear search and binary search algorithms.
    - Store products in an array for linear search and a sorted array for binary search.
4. **Analysis:**
    - Compare the time complexity of linear and binary search algorithms.
    - Discuss which algorithm is more suitable for your platform and why.

Solution:



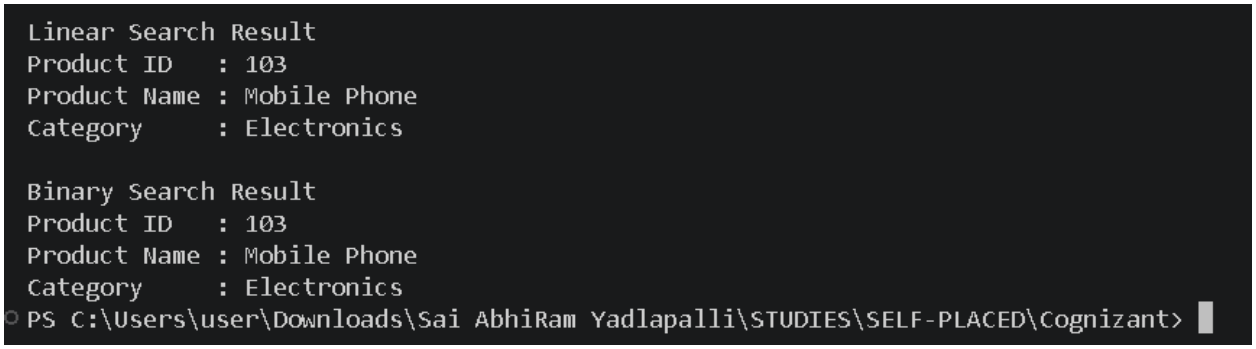
```java
import java.util.*;
class Product {
    int productId;
    String productName;
    String category;
    Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }
    void display() {
        System.out.println("Product ID   : " + productId);
        System.out.println("Product Name : " + productName);
        System.out.println("Category     : " + category);
    }
}
public class EcommerceSearch {
    static Product linearSearch(Product[] products, int targetId) {
        for (Product product : products) {
            if (product.productId == targetId) {
                return product;
            }
        }
        return null;
    }
    static Product binarySearch(Product[] products, int targetId) {
        int low = 0;
        int high = products.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (products[mid].productId == targetId) {
                return products[mid];
            }
            if (products[mid].productId < targetId) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return null;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("E-Commerce Product Search");
        System.out.print("Enter number of products: ");
        int n = sc.nextInt();
```

```
        sc.nextLine();
        Product[] products = new Product[n];
        for (int i = 0; i < n; i++) {
            System.out.println("\nProduct " + (i + 1));
            System.out.print("Enter Product ID: ");
            int id = sc.nextInt();
            sc.nextLine();
            System.out.print("Enter Product Name: ");
            String name = sc.nextLine();
            System.out.print("Enter Category: ");
            String category = sc.nextLine();
            products[i] = new Product(id, name, category);
        }
        System.out.print("\nEnter Product ID to Search: ");
        int searchId = sc.nextInt();
        Product linearResult = linearSearch(products, searchId);
        System.out.println("\nLinear Search Result");
        if (linearResult != null) {
            linearResult.display();
        } else {
            System.out.println("Product not found.");
        }
        Product[] sortedProducts = Arrays.copyOf(products, products.length);
        Arrays.sort(sortedProducts, Comparator.comparingInt(p ->
p.productId));
        Product binaryResult = binarySearch(sortedProducts, searchId);
        System.out.println("\nBinary Search Result");
        if (binaryResult != null) {
            binaryResult.display();
        } else {
            System.out.println("Product not found.");
        }
        sc.close();
    }
}
```

Output:



Suitability:

Binary Search is more Suitable since there are more than 1000 documents containing the Products details so Binary Search works Faster. Seeing the Complexity difference the Linear Search works O(n) where as Binary Search works O(log n). So its better to use Binary Search only limitation that we got is that the Elements of the Search to be in Sorted order so that the Search happens Efficiently…

Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Steps:

1. **Understand Recursive Algorithms:**
    - Explain the concept of recursion and how it can simplify certain problems.
2. **Setup:**
    - Create a method to calculate the future value using a recursive approach.
3. **Implementation:**
    - Implement a recursive algorithm to predict future values based on past growth rates.
4. **Analysis:**
    - Discuss the time complexity of your recursive algorithm.
    - Explain how to optimize the recursive solution to avoid excessive computation.

Solution:

Recursion:

It is a programming technique in which a method calls itself to solve a problem. We break a large problem into smaller sub problems until all conditions are satisfied. Recursion can simplify problems that have repetitive or hierarchical structures.

```
import java.util.Scanner;
public class Money {
    static double predictFutureValue(double currentValue, double growthRate,
int years) {
        if (years == 0) {
            return currentValue;
        }
        return predictFutureValue(
                currentValue * (1 + growthRate / 100),
                growthRate,
                years - 1
        );
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Money Tool");
        System.out.print("Current Value: ");
```

```
        double currentValue = sc.nextDouble();
        System.out.print("Annual Rate (%): ");
        double growthRate = sc.nextDouble();
        System.out.print("Years: ");
        int years = sc.nextInt();
        double futureValue = predictFutureValue(
                currentValue,
                growthRate,
                years
        );
        System.out.printf("\nPredicted Value after %d years: %.2f",
                years,
                futureValue);
        sc.close();
    }
}
```

Output:

```
Money Tool
Current Value: 100000
Annual Rate (%): 10
Years: 67

Predicted Value after 67 years: 59334857.76
PS C:\Users\user\Downloads\Sai AbhiRam Yadlapalli\STUDIES\SELF-PLACED\Cognizant>
```

Complexity:

For every year : T(n) = T(n-1) + O(1)
Time and Space Complexity will be O(n). Where as n is number of years.

Optimization:

The recursive solution can be optimized by: Iterative approach or by Direct formula

This eliminates repeated recursive calls and improves efficiency for large values of years.